



MODULE 13

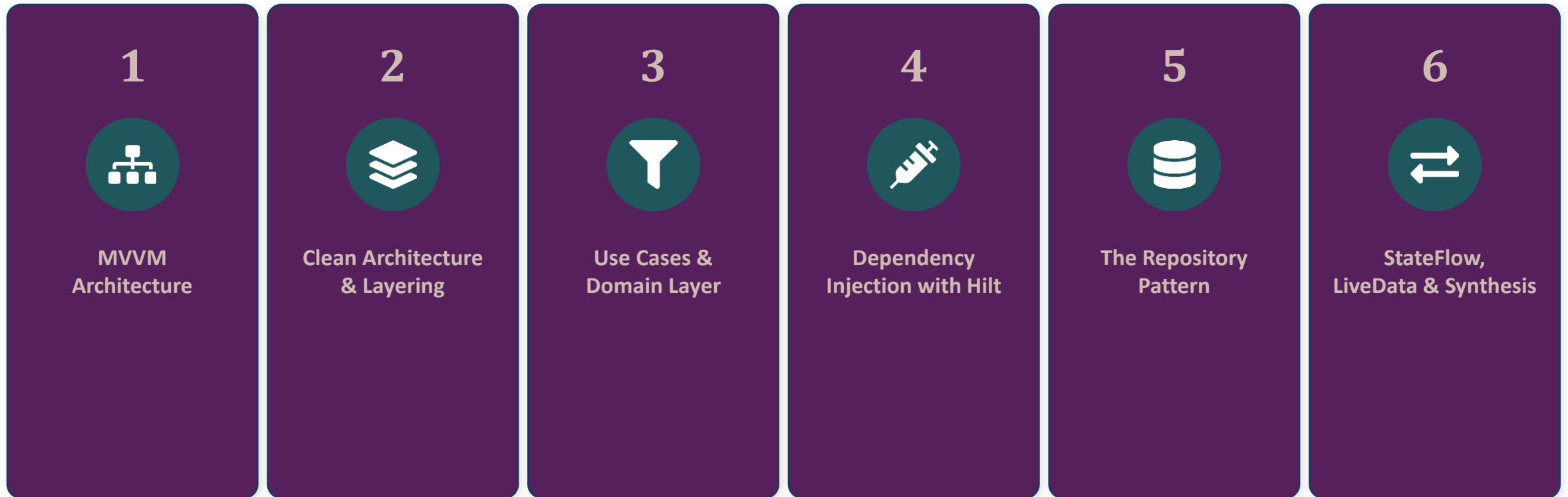
Android Architecture Patterns

Enterprise Mobile Application Development Bootcamp | AI-Integrated Edition

Phase 03 | Android Development with Kotlin

Module 13 Roadmap

Six blocks. By the end: a refactored, testable, Hilt-injected PNC Mobile Android feature.



MVVM Architecture

- The Massive Activity Problem
- Extracting the ViewModel
- the “Zero Android Framework Imports” Test



The Massive Activity Problem

```
class AccountsActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        lifecycleScope.launch {  
            val response = httpClient.get("https://api.pncmobile.com/accounts")  
            val accounts = Json.decodeFromString<List<Account>>(response.body())  
            for (account in accounts) {  
                if (account.balance < 0) showOverdraftWarning(account)    // business rule  
            }  
            recyclerView.adapter = AccountAdapter(accounts)                // UI update  
        }  
    }  
    // ...plus click listeners, error dialogs, formatting, navigation...  
}
```



ViewModels in Compose

- In many frameworks a ViewModel is simply any object that you use together with a view
- In Jetpack Compose, ViewModel is a class with very specific functionality that you inherit
- The ViewModel's job is to expose state and business logic to the UI
 - It caches its state and persists it through configuration changes such as screen rotation
 - This means that the UI doesn't have to fetch data again when navigating between activities, rotating the screen, or changing posture (such as opening/closing a foldable)
- ViewModels are used in conjunction with StateFlow



ViewModel Dependencies

- The Lifecycle Viewmodel Compose and the Lifecycle Runtime Compose artifacts are needed for ViewModel functionality
- These should be added to your app's build.gradle file

Code Bite

```
implementation  
"androidx.lifecycle:lifecycle-viewmodel-  
compose:2.8.7"
```

```
implementation  
"androidx.lifecycle:lifecycle-runtime-  
compose:2.8.7"
```



Accessing the ViewModel

- The `viewModel()` function finds the closest `ViewModelStoreOwner` and fetches the active `ViewModel` instance
 - This ensures state survives configuration and posture changes
 - (`ViewModelStoreOwner` is your host Activity, Fragment, or Navigation BackStack entry)
- It uses a fetch-or-create logic to ensure that you will always receive an instance of the `ViewModel`

Code Bite

```
@Composable
fun AccountProfileRoute(
    viewModel: AccountProfileViewModel
        = viewModel()
) {

}
```



StateFlow

- **StateFlow** is the standard and recommended way to expose business logic and screen state from a ViewModel to the UI
 - It is observable
- State changes should only be made through the ViewModel
 - Keep a private MutableStateFlow
 - Expose an immutable view of the StateFlow as a public property for the UI to observe

Code Bite

```
private val _uiState = MutableStateFlow(
    AccountProfileState
)

public uiState:
    StateFlow<AccountProfileState> =
        _uiState.asStateFlow()
```



MVVM: Extracting the ViewModel

```
class AccountsViewModel(private val repository: AccountsRepository) : ViewModel() {  
    private val _accounts = MutableStateFlow<List<Account>>(emptyList())  
    val accounts: StateFlow<List<Account>> = _accounts  
  
    fun loadAccounts() {  
        viewModelScope.launch {  
            _accounts.value = repository.fetchAccounts()  
        }  
    }  
  
    val overdraftAccounts: List<Account>  
        get() = _accounts.value.filter { it.balance < 0 }  
}
```



remember

- **remember** stores state directly in the composition tree memory
- It acts as a local cache that is retained only as long as the composable remains active in the UI hierarchy
- **Best for:** very temporary, short-lived, or local visual properties that are easy to recreate if lost
- **Examples:** an animation state, a Boolean flag tracking whether a dropdown menu or dialog is open, or an intermediate expanded/collapsed card state



rememberSaveable

- **rememberSaveable** works similarly to **remember**, but it hooks into Android's native savedInstanceState mechanism
- This allows it to serialize your state into a Bundle, preserving it even if the screen rotates or the operating system terminates the app process in the background
- **Limitations:** The data must be standard primitives (like strings, integers, booleans) or explicitly implement Parcelable
 - It is restricted by small Bundle size limits and should not handle massive objects
- **Best For:** Small items of user input or UI state that a user expects to persist during orientation changes or app interruptions
- **Examples:** Current text in an unsubmitted form field, current scroll position of a list, or the currently active navigation tab



ViewModel

- A **ViewModel** is an architectural component tied to the lifecycle of a screen (like an Activity or Navigation graph destination) rather than a single Composable
- It acts as the "brain of the UI," managing heavy business logic and keeping data intact across configuration changes like screen rotations
- **Note on Process Death:** A standard ViewModel *does not* automatically survive process death
 - You must inject a SavedStateHandle into it to backup crucial pieces of state
- **Best For:** Complex data structures, handling network requests, querying a database, or sharing state between multiple composable components on a screen
 - ViewModels also move testing logic out of the UI layer, enabling fast local JVM unit tests
- **Examples:** A user profile object fetched from an API, search results from a database repository, or managing complex screen state via state management architectures (like a MutableStateFlow of your UI state)



remember vs. rememberSaveable vs. ViewModel

Feature/Behavior	remember	rememberSaveable	ViewModel
Survives recomposition?	Yes	Yes	Yes
Survives screen rotation?	No	Yes	Yes
Survives process death	No	Yes	No (Yes w/ SavedStateHandle)
Data type support	Any object type	Only Bundle/Parcelable	Any object type
Primary scope	Local/Ephemeral UI	Small, persistent UI state	Screen-level/Business logic



Block 1 - MVVM Architecture

Skill Check ✓

- Name three distinct responsibilities crammed into the Massive Activity example.
- What's the literal, checkable test for whether business logic has leaked into an Activity?
- In the MVVM version, what's left for the Activity to actually do?



Clean Architecture & Layering

- The Dependency Rule
- Presentation, Domain & Data Layers
- Mapping Layers onto an Android Project



Clean Architecture

- Clean Architecture is a software design philosophy introduced by Robert C. Martin ("Uncle Bob")
- It organizes code into concentric layers to keep core business logic completely independent of external concerns like databases, user interfaces, and frameworks
- Its principles include:
- **Inward Flow:** Source code dependencies can only point inward
- **Inner Isolation:** The innermost layers know nothing about the outer layers. They do not mention functions, classes, or variables from the outside
- **Outer Adaptability:** Outer layers can change frequently (like swapping a SQL database for a NoSQL store or changing a web framework), but the inner business logic remains untouched



The Dependency Rule

Presentation

Composables, ViewModels - what the user sees and interacts with

depends on ↓

Domain

Business rules, use cases, models - the core of the app, no framework imports

depends on ↓

Data

Repositories, API clients, Room DAOs - where data actually comes from

Dependencies point downward only. Domain never imports from Presentation or Data.



Mapping Layers onto a Real Android Project



presentation/

AccountListScreen.kt, AccountsViewModel.kt



domain/

Account.kt, AccountsRepository.kt (interface only), GetAccountsUseCase.kt



data/

AccountsRepositoryImpl.kt, AccountsApiService.kt, AccountDao.kt

Notice: the domain layer defines AccountsRepository as an interface. The data layer provides AccountsRepositoryImpl. This inversion is what makes the dependency point inward - Domain doesn't know Data exists.



Block 2 - Clean Architecture & Layering

Skill Check ✓

- State the Dependency Rule in one sentence.
- Which layer is allowed to import Jetpack Compose, and which layer must never import it?
- Why does the Domain layer define AccountsRepository as an interface rather than a concrete class?



Use Cases & Domain Layer

- What a Use Case Actually Is
- One Job, One Class
- Where Business Rules Actually Live



What is a Use Case?

- A **use case** is a single class that contains application-specific business logic for one specific task or user action
- It acts as the bridge between your outer layers (like the UI or web controllers) and your core business rules and entities



Use Case Key Responsibilities

- **Orchestration:** Coordinates the flow of data to and from domain entities and calls repositories to fetch or save data
- **Single Responsibility:** Handles one specific operation the system can perform (e.g., LoginUserUseCase, SendPaymentUseCase)
- **Isolation:** Knows *what* the application needs to do, but does not know *how* outer details like databases, frameworks, or user interfaces work



Use Case Characteristics

- **Inward Dependencies:** It sits in the application/domain layer and depends only on abstractions (interfaces), allowing outer technologies to change without affecting the core logic
- **Synonyms:** Often referred to as an **interactor**
- **Testability:** Because it has no external dependencies on UI frameworks or databases, it can be tested quickly via pure unit tests



Use Case Examples

```
class GetAccountsUseCase(private val repository: AccountsRepository) {  
    suspend operator fun invoke(): List<Account> {  
        return repository.fetchAccounts()  
    }  
}
```

```
class TransferFundsUseCase(  
    private val repository: AccountsRepository,  
    private val eligibilityService: TransferEligibilityService  
) {  
    suspend operator fun invoke(amount: Double, from: Account, to: Account): Result<Unit> {  
        val eligible = eligibilityService.canTransfer(amount, from)  
        if (!eligible) return Result.failure(InsufficientFundsException())  
        repository.transfer(amount, from, to)  
        return Result.success(Unit)  
    }  
}
```



One Job, One Class



Independently testable

A use case's test only needs to fake exactly the dependencies it declares - nothing more



Independently reusable

GetAccountsUseCase can be called from a ViewModel, a widget, or a background sync job identically



Self-documenting

The class list of a domain/ package IS a list of everything the app can do, in plain English



Block 3 - Use Cases & Domain Layer

Skill Check ✓

- What does the operator `invoke()` convention let you do when calling a use case?
- How is a use case more granular than Module 7's Service layer?
- Name one concrete benefit of one-job-one-class use cases beyond just organization.

